

# FocusFlow

Application mobile de productivité — Méthode Pomodoro

---

|         |                                                                                                                                       |
|---------|---------------------------------------------------------------------------------------------------------------------------------------|
| Classe  | L3GLSIA                                                                                                                               |
| Membres | El Hadj Boubacar Mamadou Demba Bathily<br>Kadiatou Sadio Diallo<br>Serigne Saliou Mbacké Niang<br>Mamadou Diabou Gning<br>Ndeye Gueye |

## 1. Présentation du Projet

---

**FocusFlow** est une application mobile de productivité axée sur la méthode Pomodoro. Elle combine un minuteur de sessions de travail et de pauses avec un gestionnaire de tâches intégré, dans l'objectif d'aider l'utilisateur à maintenir durablement sa concentration.

### Fonctionnalités clés :

- Minuteur Pomodoro interactif avec arc de progression animé
- Association d'une tâche spécifique à chaque session de travail
- Système de *feedback* en fin de session influençant les statistiques
- Diffusion de sons d'ambiance (pluie, océan, forêt...) pour l'immersion
- Tableau de bord analytique : temps de focus hebdomadaire et taux de succès

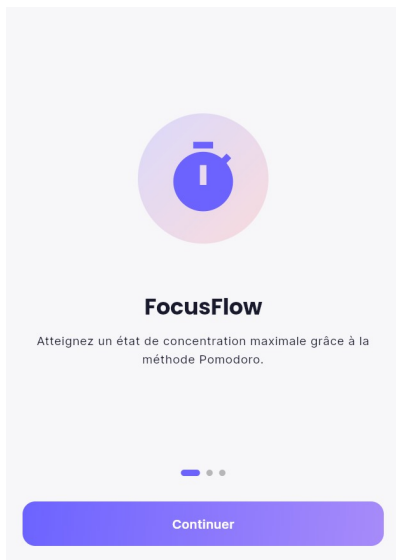


Fig. 1 — Écran d'accueil (Onboarding) Présentation du concept avec indicateurs de progression en dots.



Fig. 2 — Dashboard principal Résumé du jour, tâche en cours et liste récente. Navigation BottomBar à 4 onglets.

## 2. Choix Techniques

L'architecture est scindée en deux couches pour garantir la séparation des responsabilités et la maintenabilité du code.

### Frontend — Flutter & Dart

- **Flutter / Dart** — UI native multi-plateforme (iOS & Android)
- **provider** (MultiProvider) — gestion de l'état global via AuthProvider, TimerProvider, TaskProvider, SettingsProvider
- **shared\_preferences** — persistance locale du token JWT et des préférences utilisateur
- **audioplayers** — sons d'ambiance en boucle
- **fl\_chart** — graphiques dynamiques sur l'écran Statistiques

### Backend — Node.js & Express

- **Node.js + Express.js** — API REST légère
- **JWT + bcryptjs** — authentification stateless et hachage des mots de passe
- **SQLite + Prisma** — base de données locale avec ORM typé et migrations de schémas
- **cors / dotenv** — gestion des requêtes cross-origin et sécurisation des variables d'environnement

## 3. Captures d'écran

Les captures suivantes illustrent les principaux écrans en fonctionnement réel.

### 3.1 Timer Pomodoro

Afin de maximiser la concentration de l'utilisateur, l'application adopte une approche stricte de la méthode Pomodoro reposant sur trois actions clés : la réinitialisation, la pause et l'achèvement anticipé.

Le bouton de **réinitialisation** permet d'annuler un chronomètre lancé par erreur sans polluer la base de données. La **pause**, quant à elle, n'est plus une simple suspension du temps : son activation nécessite une confirmation et entraîne l'arrêt immédiat et définitif de la session, automatiquement enregistrée comme un « Échec » dans la base de données. Cette session fera chuter le taux de succès global et apparaîtra dans l'historique récent, mais ses minutes ne seront pas comptabilisées dans le graphique du temps de focus — réservé au travail productif. Enfin, l'**achèvement anticipé** permet de clôturer victorieusement une session si la tâche est terminée plus tôt que prévu. Ce système exigeant garantit que les statistiques reflètent exclusivement de véritables blocs de travail ininterrompus.

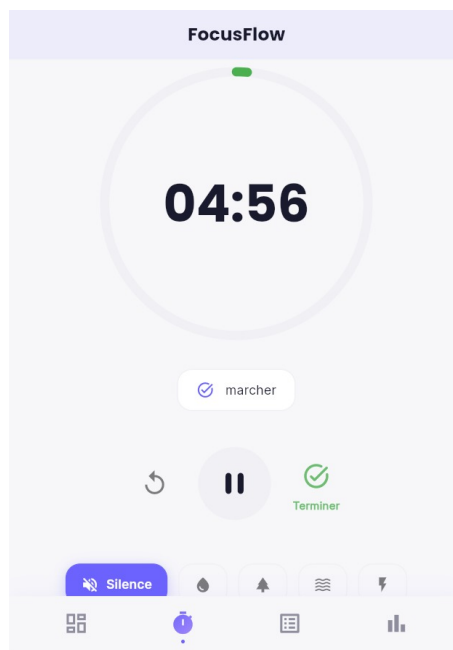


Fig. 3 — Timer Pomodoro en cours de session

L'arc circulaire indique la progression de la session. Le temps restant est affiché en grand. La tâche liée apparaît en carte sous le cercle. Les trois contrôles (réinitialiser / pause / terminer) sont visibles en bas, et le sélecteur de sons d'ambiance (Silence, Goutte, Forêt, Vagues, Éclair) est accessible via des icônes au-dessus de la barre de navigation.

### 3.2 Gestion des Tâches

L'écran des tâches propose des filtres rapides (Toutes, En cours, Complétées, Haute priorité) et une bottom sheet de création glissant depuis le bas, permettant de saisir le titre, une description optionnelle et de choisir la priorité via des chips colorés.

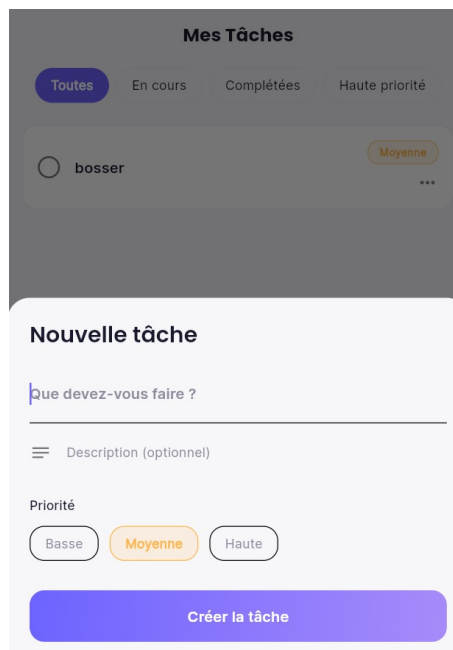


Fig. 4 — Écran Tâches + Bottom Sheet de création

Chips de filtrage horizontal en haut. Chaque tâche affiche un badge de priorité coloré. La bottom sheet permet de créer une nouvelle tâche sans quitter l'écran.

### 3.3 Statistiques & Historique

Le tableau de bord analytique présente les minutes de focus cumulées et le taux de succès global, suivi d'un graphique d'activité sur les 7 derniers jours. L'historique liste chaque session passée avec sa date et son résultat.



Fig. 5 — Statistiques Métriques Minutes de Focus et Taux de succès. Graphique linéaire (fl\_chart) sur 7 jours.

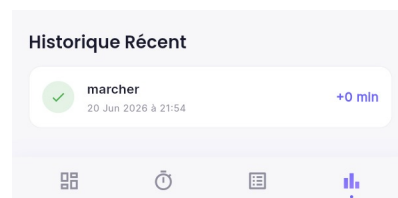


Fig. 6 — Historique récent Liste des sessions avec date et heure. Icône verte = session complétée avec succès.

## 4. Difficultés Rencontrées

- 
- **Synchronisation Timer ↔ État global** — Faire interagir le minuteur avec la base de données sans provoquer de rechargements d'interface ou figer le compte à rebours. Gérer la suspension du flux pour recueillir le *feedback* de l'utilisateur en fin de session a nécessité un soin architectural particulier.
  - **Audio sur Flutter Web** — Sous Chrome, la lecture de certains formats audio retournait l'erreur *MEDIA\_ELEMENT\_ERROR: Format error (Code: 4)* liée aux codecs non supportés.
  - **Conflits Hero Tags** — Lors de la migration vers une navigation par *IndexedStack*, une exception « multiple heroes sharing the same tag » bloquait le rendu, causée par la duplication du *FloatingActionButton* dans plusieurs écrans actifs simultanément.
  - **Dépréciation de composants Flutter** — De nombreux avertissements ont pollué la compilation : *withOpacity* → *withValues*, *activeColor* → *activeTrackColor*, migration vers Material 3, etc.

## 5. Solutions Proposées

---

- **Refactoring du flux Pomodoro** — Le *TimerProvider* a été restructuré pour capturer l'état *needsFeedback*. À zéro, le flux est suspendu le temps d'afficher la boîte de dialogue. L'appel API n'est déclenché qu'après réception du statut utilisateur.
- **Externalisation de l'audio** — Les URLs des sons pointent vers des fichiers hébergés externement via requêtes HTTP directes, contournant les limitations de codecs locaux de Chrome.
- **Nettoyage de la structure Scaffold** — Le FAB du conteneur principal a été supprimé. Chaque sous-écran gère son propre *FloatingActionButton* avec un tag unique, éliminant l'exception Hero.
- **Migration Material 3** — Remplacement de *accentColor*, adaptation de *withOpacity* et de *CardThemeData* à la dernière syntaxe Flutter — résultat : 0 erreur de compilation.

## 6. Conclusion

---

Le projet **FocusFlow** remplit son objectif : proposer un outil ludique, performant et visuellement soigné pour favoriser la concentration. Le système Pomodoro strict — réinitialisation, pause définitive et achèvement anticipé — garantit des statistiques fiables reflétant uniquement de véritables blocs de travail productifs.

Les choix techniques retenus — Flutter pour le rendu natif et Node.js/Prisma pour un backend souple — posent des fondations solides pour des évolutions futures telles que le partage de sessions entre utilisateurs, la synchronisation cloud multi-appareils ou des thèmes visuels débloquables selon les performances.